NAVSAHYADRI EDUCATION SOCIETY'S GROUP OF INSTITUTIONS, PUNE
Department of Computer Engineering
Academic Year 2026-27

JARVIS OS VOICE ENABLED DESKTOP AI ASSISTANT AND PC AUTOMATION SYSTEM

PROJECT SYNOPSIS

OF MAJOR PROJECT

Submitted to Savitribai Phule Pune University in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING

Computer Engineering

SUBMITTED BY

[STUDENT NAME]

University Roll No. / PRN: [ADD NUMBER]

Class Roll No.: [ADD NUMBER] Batch: [ADD BATCH]

Under the guidance of

[PROJECT GUIDE NAME]



NAVSAHYADRI COLLEGE OF ENGINEERING
PUNE

TABLE OF CONTENTS

Introduction 3

Rationale and Objectives 4

Proposed System and Scope 5

Literature Review 6

Feasibility Study 7

Methodology and Planning of Work 8

Facilities Required for Proposed Work 9

Expected Outcomes 9

References 10

Introduction

JARVIS OS is a Python desktop assistant prototype that brings common computer tasks and conversational responses into a terminal interface and a browser based HUD. Users can type commands or use voice input where the required browser or speech components are available. A shared router sends requests to built in skills or, for open ended conversation, an optional language model provider. The interface returns text and can use browser speech or operating system text to speech for spoken replies.

The current project includes skills for application and website launching, file search and selected file operations, system information, calculations, notes, timers, weather and web knowledge, scheduled commands, and selected Windows controls. Settings and several user data items use local JSON files. Optional providers include a local Ollama endpoint and a configured cloud model service. The project is an application for an existing desktop operating system; it does not replace the operating system itself.

Problem Definition

Everyday desktop tasks are spread across separate applications and command interfaces. A language model can converse, but text generation alone cannot safely confirm that a local action was completed. The project therefore addresses the need for one natural language interface that routes supported requests to bounded tools, checks the tool result, and reports success or failure accurately. The prototype also needs clear limits for voice privacy, local computer access, provider outages, and actions that can change or delete user data.

Rationale and Objectives

A desktop assistant can reduce the steps needed for common tasks such as opening applications, checking system information, setting timers, or finding files. Jarvis combines deterministic skills for known actions with an optional language model for general conversation. This design gives the assistant a practical role while keeping computer actions in explicit, testable functions.

The project is relevant to human-computer interaction and applied artificial intelligence because it combines natural language, speech interfaces, tool integration, and local desktop control. Its value will be assessed through task scenarios and safety checks rather than through claims that the prototype matches commercial assistants.

1. Design a modular desktop assistant with terminal and browser based interfaces that share one command routing flow.
2. Implement natural language routing to focused skills for common computer, information, and productivity tasks.
3. Integrate optional language model and speech providers while making provider availability and cloud data use clear to the user.
4. Apply confirmation and validation to selected sensitive actions, then evaluate routing accuracy, action results, failures, and usability with repeatable scenarios.

Proposed System and Scope

Jarvis accepts typed or spoken requests, parses command aliases and natural language, and routes them through a shared Python Router. Deterministic skills perform supported tasks; the conversation module calls a configured language model when a request needs open ended text. Selected destructive actions request confirmation, and application launches use verification where supported. The HUD and terminal display the resulting message.

Table 1 JARVIS OS Architecture Components

Component	Role in the prototype
Interfaces	Python terminal REPL and browser HUD; browser microphone and speech output are optional.
Command routing	Parser and shared Router interpret commands and select a skill or conversational response.
Skills and tools	Modules support applications and websites, files, selected PC controls, system status, maths, notes, timers, weather, and web knowledge.
Conversation and providers	ChatBrain provides local responses and can call a configured LLM provider; the model does not itself verify desktop actions.
State and safeguards	Local JSON stores selected settings and data. Confirmation and path checks guard selected destructive file actions; the local web bridge is intended for single-user loopback use.

Scope and Limitations

The project focuses on a local single user desktop prototype, with Windows controls as the primary computer-control target. Local commands can operate without a cloud LLM, while online weather, web lookup, speech recognition, or cloud model use may require an internet connection and can send request data outside the device. The user must be told which provider is active before cloud processing is used. The current bridge should not be exposed to a network or used by multiple people until server-side authorization and per-session state are implemented. Autonomous unrestricted shell execution, a mobile client, and a production multi-user service are outside the current scope.

Literature Review

Recent research treats language models as components that can interact with tools, rather than as stand alone sources of verified action. Yao et al. introduce ReAct, which combines language based task reasoning with actions that gather information from external sources. This supports a design in which Jarvis routes concrete tasks to observable skills and uses their results when answering [1].

Schick et al. present Toolformer, a method for deciding when to call APIs, what arguments to pass, and how to use returned results. Jarvis currently uses predefined skills rather than learned API selection, but the work supports keeping tool interfaces narrow and explicit [2].

Radford et al. study speech recognition trained on large scale weak supervision and evaluate its generalisation across speech tasks. The work is relevant to Jarvis voice input, which should be treated as a distinct provider component and evaluated separately from command routing [3].

Ruan et al. introduce ToolEmu, a sandbox based approach for finding risks in language model agents that use tools. Their work motivates testing failure cases and high impact actions before expanding Jarvis controls. Together, these studies suggest that tool boundaries, speech quality, and safety evaluation should be part of the assistant design, not added after deployment [4].

Feasibility Study

Technical feasibility is supported by the existing Python modules, standard library HTTP bridge, browser based HUD, and optional voice and system monitoring packages. The current skill architecture already covers a bounded set of desktop and information tasks. A local Ollama server can provide model inference without sending chat text to a cloud provider; a cloud provider remains optional and may require a key, internet access, and consent. Windows is the primary target for PC control, while terminal, web, and conversation features can be tested separately from hardware actions.

Operationally, the prototype is suitable for a single user on a local computer. Some natural language file and power requests ask for confirmation, while typed PC control commands are treated as explicit requests. The web API still needs server-side authorization and session isolation before broader use. Economic feasibility is reasonable for a student project because the code uses Python and web technologies with open source libraries; optional cloud inference or online services may add usage costs. The work can be completed incrementally by prioritising the reliable local core, then voice and model integrations, followed by testing and documentation. Exact dates should be set with the project guide.

Methodology and Planning of Work

This is an applied software engineering project. Development will proceed through requirements analysis, modular design, incremental implementation, and scenario based evaluation. The test environment will use mock providers and isolated temporary files where possible; hardware actions will remain opt in.

1. Review user requirements and define supported tasks, privacy boundaries, confirmation rules, and measurable acceptance criteria.
2. Design the command flow, skill interfaces, provider configuration, local data model, and terminal and HUD responsibilities.
3. Implement and refine the parser, Router, desktop skills, browser interface, optional LLM provider, and voice pathway in small increments.
4. Test normal, ambiguous, offline, and failure cases for each skill. Check that actions return verified results and that protected actions cannot bypass confirmation or authorization.
5. Evaluate representative scenarios such as opening an application, setting a timer, retrieving system status, answering a general question, and cancelling a protected action. Record task completion, routing errors, unsupported success claims, and response time.

The existing unit tests provide a starting point for regression checks. Hardware dependent checks will be separated from routine automated tests so evaluation does not accidentally change files, settings, or running applications.

Facilities Required for Proposed Work

Development requires a Windows computer, Python 3, a modern web browser, and an editor. A microphone and speakers are needed to evaluate voice input and output; an internet connection is needed for online search, weather, server speech recognition, or cloud model calls. Ollama is optional for local model inference, and optional Python packages support microphone capture and system metrics.

Expected Outcomes

The expected outcome is an integrated Jarvis desktop assistant prototype with a terminal and browser interface, a shared command router, modular skills, optional language model and voice providers, and guarded access to selected computer functions. The project will document its local and cloud data paths and evaluate representative tasks, provider failures, and protected actions. Accuracy, latency, and task completion results will be reported after testing; no performance results are claimed in this synopsis.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[2] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in Adv. Neural Inf. Process. Syst., vol. 36, 2023. [Online]. Available: https://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in Proc. 40th Int. Conf. Mach. Learn., Proc. Mach. Learn. Res., vol. 202, pp. 28492–28518, 2023. [Online]. Available: https://proceedings.mlr.press/v202/radford23a.html

[4] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2024. [Online]. Available: https://openreview.net/forum?id=GEcwtMk1uA